

LogiTrack — Logistics & Supply Chain Intelligence Platform

1. Opening — 1 Minute

If the interviewer asks:

“Tell me about your project.”

Start like this:

“Sure. My project is called **LogiTrack — Logistics and Supply Chain Intelligence Platform**.

The main idea behind this project is to build an end-to-end data platform for a logistics company where data comes from multiple sources such as orders, customers, drivers, vehicles, warehouses, GPS tracking, weather, traffic, and delivery events.

Instead of analyzing each dataset separately, I designed a pipeline that integrates these sources, processes both batch and simulated real-time data, stores the data in different layers, performs analytics and machine learning, and finally presents the results through Power BI dashboards.

Technically, the architecture uses Python for ingestion and analytics, Kafka for event streaming, HDFS/S3 as the data-lake layer, Spark/PySpark for distributed processing, Hive for data organization, a dimensional data warehouse for analytics, dbt for transformations, Airflow for orchestration, Scikit-learn for machine learning, Power BI for visualization, and Prometheus with Grafana for monitoring.

The ultimate goal is to convert raw logistics data into actionable business intelligence — for example, identifying delivery delays, analyzing driver and vehicle performance, understanding warehouse workload, studying the effect of weather and traffic, and predicting future demand and delivery delays.”

Then pause.

This gives them the **big picture first**.

2. Business Problem — 2 Minutes

Then explain **why you built it**.

"The business problem I focused on is that a logistics company generates data from many different operational systems.

For example, the order system contains customer and order information. The fleet system contains vehicle information. GPS systems continuously generate location data. Warehouses generate operational data. There can also be external data such as weather and traffic.

If these datasets are isolated, it becomes difficult for management to answer basic questions."

Then give examples:

"For example:

- Which regions have the highest delivery delays?
- Which drivers have high workload but poor performance?
- Which vehicles are most active?
- Which warehouses handle the highest order volume?
- Does heavy rainfall increase delivery delays?
- Does traffic affect delivery performance?
- Which customers generate the highest order value?
- What will the expected demand be next month?
- Which current deliveries are at higher risk of being delayed?"

Then:

"So I designed LogiTrack as a centralized data and analytics platform that brings these datasets together."

Business value

Explain:

"From a business perspective, the platform is mainly focused on five areas:

1. Delivery intelligence
2. Fleet and driver intelligence
3. Warehouse intelligence
4. Customer and financial analytics
5. Predictive analytics"

Then explain each briefly.

Delivery

"For delivery, I calculate metrics such as total deliveries, on-time delivery rate, delay rate, average delay and maximum delay."

Fleet

"For fleet, I analyze GPS observations, vehicle activity, speed, status and low-signal observations."

Warehouse

"For warehouses, I analyze workload, order value, delivery performance and regional differences."

Customer

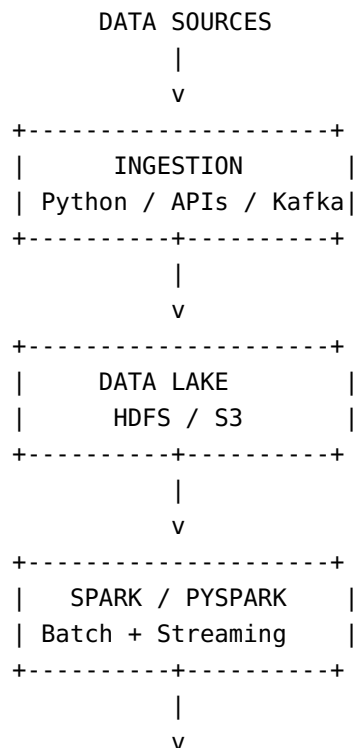
"For customers, I analyze order frequency, customer value, repeat behavior and delivery experience."

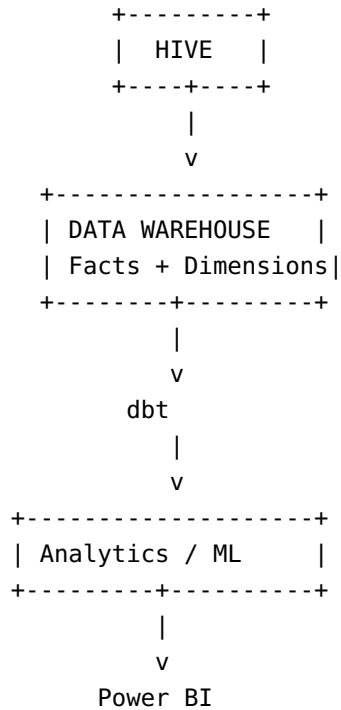
Prediction

"Finally, I use historical data for baseline demand forecasting and delivery-delay prediction."

3. Overall Architecture — 2 Minutes

Now draw this mentally on the whiteboard:





Airflow → Orchestration
Prometheus + Grafana → Monitoring

Say:

“The architecture is layered because I don't want raw data, transformation logic, analytics logic and visualization logic all mixed together.

Each layer has a specific responsibility.”

Then explain the layers one by one.

4. Data Sources — 1 Minute

“The first layer consists of the source datasets.”

Your main datasets are:

```
orders.csv
customers.csv
drivers.csv
vehicles.csv
warehouses.csv
```

```
gps_tracking.csv
weather.csv
traffic.csv
roads.csv
delivery_updates.csv
```

Explain what each represents.

Orders

```
order_id
customer_id
warehouse_id
driver_id
vehicle_id
order_date
order_value
delivery_distance_km
promised_delivery_time_min
actual_delivery_time_min
```

GPS

```
vehicle_id
timestamp
latitude
longitude
speed_kmph
heading
vehicle_status
signal_quality
```

Weather

```
date
location
temperature
humidity
rainfall
weather_condition
```

Traffic

```
timestamp
location
traffic_score
traffic_level
average_speed
```

Then say:

“The operational datasets are primarily synthetic for this portfolio project, while contextual datasets can include public or API-based data. For real-time data, I simulate streaming events because I don't have access to a production logistics company's live GPS infrastructure.”

That sentence is **very important**.

5. Ingestion Layer — 2 Minutes

Now explain Python ingestion.

“The next layer is ingestion. I separated ingestion into batch, API and streaming ingestion.”

Batch ingestion

You have:

```
ingestion/batch/orders_ingestion.py
ingestion/batch/customer_ingestion.py
ingestion/batch/warehouse_ingestion.py
```

Explain:

“The batch ingestion scripts read source files, validate the expected columns, standardize the data and remove duplicate records where necessary before writing the data into the raw layer.”

For example:

```
df = pd.read_csv(input_file)

required_columns = [...]
```

```
missing = set(required_columns) - set(df.columns)

if missing:
    raise ValueError(f"Missing columns: {missing}")

df = df.drop_duplicates()
```

Explain:

"The important thing is that I don't blindly load the data. I first validate the schema."

6. API Ingestion — 1 Minute

You have:

```
weather_api.py
traffic_api.py
maps_api.py
```

Explain:

"For external data, I created API ingestion modules. For example, the weather module can call a weather API, normalize the response into the project's weather schema and save it into the raw layer."

Credentials are externalized:

```
WEATHER_API_KEY
TRAFFIC_API_KEY
MAPS_API_KEY
```

Say:

"I deliberately don't hardcode API keys. They are supplied through environment variables."

This is a nice **security point**.

7. Kafka — 2 Minutes

Now this is where the interviewer may become interested.

Say:

"For real-time events, I use Apache Kafka."

Your topics:

```
orders
gps_tracking
delivery_updates
traffic_updates
weather_updates
```

Explain Kafka simply first:

"Kafka acts as a distributed event-streaming platform between producers and consumers. Producers publish events into topics, and consumers read those events independently."

Draw:

```
GPS System
  |
  v
GPS Producer
  |
  v
Kafka
  |
+-----> GPS Consumer
  |
+-----> Spark Streaming
  |
+-----> Monitoring
```

Example GPS event

```
{
  "vehicle_id": "V101",
  "timestamp": "2026-08-31 10:30:00",
  "latitude": 13.0827,
  "longitude": 80.2707,
  "speed_kmph": 42
}
```

Then explain the producer:

"The GPS producer reads GPS records and publishes them to Kafka. I use `vehicle_id` as the message key because events belonging to the same vehicle should logically remain associated with that vehicle."

You can mention:

```
producer.send(  
    "gps_tracking",  
    key=vehicle_id,  
    value=event  
)
```

And:

"I also configured reliability-oriented producer settings such as `acks=all` and retries."

Why Kafka?

If they ask:

"Why not directly send data to Spark?"

Answer:

"Kafka decouples producers and consumers. The GPS system doesn't need to know who is consuming the data. Multiple downstream systems can independently consume the same events. Kafka also provides buffering, partitioning and replay capabilities."

8. Storage — HDFS / S3 — 1 Minute

Then:

"After ingestion, I separate the raw and processed data."

Conceptually:

```
HDFS / S3  
  
raw/  
  orders
```

```
customers
drivers
vehicles
warehouses
gps
weather
traffic

processed/
  orders_clean
  gps_clean
  weather_clean
  delivery_features
```

Explain HDFS:

"HDFS is useful for distributed storage in a Hadoop ecosystem. The idea is that large files can be distributed across nodes rather than relying on one machine."

Then S3:

"For cloud architecture, I also provide an S3 upload implementation because object storage is commonly used as the data-lake layer in AWS architectures."

Important:

"In this portfolio implementation, these are supported architecture patterns; I don't claim that the entire platform is running as a production AWS deployment."

9. Spark / PySpark — 2 Minutes

Now explain the processing layer.

"Apache Spark is the main processing engine in my architecture."

Why Spark?

"Because logistics data can become very large, especially GPS and streaming events. Spark provides distributed processing and allows the same ecosystem to support batch and streaming workloads."

Your batch jobs:

```
clean_orders.py
clean_gps.py
clean_weather.py
create_features.py
```

Explain order cleaning:

```
Raw Orders
  ↓
Schema validation
  ↓
Type conversion
  ↓
Null handling
  ↓
Duplicate handling
  ↓
Derived metrics
  ↓
Clean Orders
```

Important derived field:

```
delivery_delay_min
```

Formula:

```
actual_delivery_time_min
-
promised_delivery_time_min
```

Then:

```
is_delayed = delivery_delay_min > 0
```

And:

```
delivery_per_km_value
=
order_value / delivery_distance_km
```

with zero-distance protection.

10. Spark Streaming — 1.5 Minutes

Then:

“For streaming, I use Spark Structured Streaming.”

Architecture:

```
Kafka
|
v
Spark Structured Streaming
|
+---- Cleaning
|
+---- Enrichment
|
+---- Aggregation
|
v
Real-time outputs
```

Explain:

“The GPS stream reads events from Kafka, applies a defined schema, transforms the incoming data and produces processed streaming output.”

Important interview point:

“One limitation I explicitly handled is that I don't artificially join a GPS observation to an order just because both contain a vehicle ID. In a real system, that would require a proper event-time and stateful join with time windows and potentially additional delivery-state information.”

This sounds **very strong technically**.

11. Hive — 1 Minute

Then:

"After processing, I use Hive as the SQL/table organization layer over the Hadoop-oriented data environment."

Tables:

```
orders
customers
drivers
vehicles
warehouses
gps_tracking
weather
traffic
```

Hive allows analysts to query the processed data using SQL.

Example:

```
SELECT
    region,
    COUNT(*) AS total_orders,
    AVG(order_value) AS avg_order_value
FROM orders
GROUP BY region;
```

Then:

"Hive is therefore useful for SQL-based analysis while Spark handles heavier processing."

12. Data Warehouse — 2 Minutes

This is very important for a **data-role interview**.

Say:

"For business analytics, I designed a dimensional warehouse model."

Dimensions:

```
dim_customer
dim_driver
```

```
dim_vehicle
dim_warehouse
dim_location
dim_date
```

Facts:

```
fact_orders
fact_delivery
fact_gps
fact_traffic
```

Then explain **grain**.

“One of the most important design decisions is defining the grain of each fact table.”

For example:

```
fact_orders
→ one row per order

fact_delivery
→ one row per order-level delivery outcome

fact_gps
→ one row per GPS observation

fact_traffic
→ one row per traffic observation
```

Then explain why grain matters:

“If I incorrectly join an order-level table with a GPS observation table, one order could match thousands of GPS records and my revenue or order count could be duplicated. So I explicitly define the grain before performing analytical joins.”

This is one of the **best answers you can give**.

13. dbt — 1.5 Minutes

Then:

"After the warehouse layer, I use dbt for modular SQL transformations."

Three layers:

```
Staging
  ↓
Intermediate
  ↓
Marts
```

Staging

```
stg_orders
stg_gps
stg_weather
stg_traffic
```

Purpose:

"Standardize source data."

Intermediate

```
int_delivery
int_routes
int_driver_performance
```

Purpose:

"Combine datasets and apply reusable business logic."

Marts

```
delivery_mart
logistics_mart
warehouse_mart
executive_mart
```

Purpose:

"Create business-ready datasets for analytics and Power BI."

Then explain dbt advantage:

“Instead of putting all transformation logic into one huge SQL script, dbt allows me to modularize transformations, test them and maintain dependencies.”

Mention:

```
unique_orders.sql  
valid_delivery_status.sql
```

14. Airflow — 1.5 Minutes

Then:

“Airflow is responsible for orchestration rather than data processing itself.”

This distinction is important.

Say:

“Spark processes the data. dbt transforms the data. Airflow controls when these tasks run and in what order.”

Example:

```
Ingestion  
  ↓  
Spark Cleaning  
  ↓  
Feature Engineering  
  ↓  
dbt  
  ↓  
Analytics  
  ↓  
Tests
```

Airflow DAG:

```
Task 1: Ingest  
  ↓
```

```
Task 2: Clean
    ↓
Task 3: Transform
    ↓
Task 4: dbt run
    ↓
Task 5: dbt test
```

If they ask why Airflow:

“Because data pipelines have dependencies. For example, I cannot run the delivery mart before the cleaned delivery data exists. Airflow allows me to define and monitor those dependencies.”

15. Analytics — 1 Minute

Then:

“Once the curated data is available, I perform both SQL and Python analytics.”

SQL areas:

```
KPI analysis
Delivery analysis
Customer analysis
Driver analysis
Warehouse analysis
```

Python:

```
EDA
Feature engineering
Route analysis
```

Examples:

“I calculate regional delivery performance, customer value, driver workload, warehouse workload, traffic patterns and weather-related delivery performance.”

16. Machine Learning — 2 Minutes

You have two main ML use cases.

A. Demand Forecasting

Say:

“The first ML use case is demand forecasting.”

Target:

daily_order_count

Features:

day_of_week
month
day_of_month
trend
seasonal indicators

Model:

Linear Regression

Evaluation:

MAE
RMSE
 R^2

Important:

“I used a chronological train-test split rather than randomly shuffling time-series observations, because random splitting can leak future information into the training set.”

Very good interview point.

B. Delivery Delay Prediction

Then:

“The second ML use case is delivery-delay prediction.”

Target:

is_delayed

Features can include:

delivery_distance
promised_delivery_time
region
weather
driver characteristics
warehouse characteristics

Model:

Logistic Regression

Metrics:

Accuracy
Precision
Recall
F1
ROC-AUC

Then the important part:

“I was particularly careful about target leakage. For example, actual delivery time, delivery delay and the final delivery outcome cannot be used as features when I'm trying to predict that same outcome.”

If they ask why Logistic Regression:

“Because this is a baseline binary classification problem, Logistic Regression is simple, interpretable and provides probabilities. For a production system, I would evaluate stronger models such as gradient boosting and compare them using appropriate validation.”

17. Power BI — 1.5 Minutes

Then:

“The final business-facing layer is Power BI.”

Main dashboard areas:

Executive Overview
Delivery Operations
Fleet Intelligence
Warehouse Intelligence
Financial Analysis

Executive

KPIs:

Total Orders
Total Order Value
Average Order Value
Delivery Rate
Delay Rate

Delivery

On-Time Rate
Delay Rate
Delay Severity
Regional Performance
Weather Impact

Fleet

Vehicle Activity
GPS Activity
Average Speed
Driver Performance

Warehouse

Warehouse Workload
Order Value
Delivery Performance
Regional Comparison

Financial

Order Value
Customer Value
Regional Value
Value Concentration

Then:

"I designed the dashboard layer to consume curated analytical data rather than making Power BI perform all the raw data transformations."

18. Monitoring — 1 Minute

Then:

"I also included an observability layer using Prometheus and Grafana."

Prometheus:

"collects metrics."

Grafana:

"visualizes those metrics."

Examples:

CPU
Memory
Disk
Network

```
System uptime
Target health
```

For a production implementation, I would additionally monitor:

```
Kafka consumer lag
Spark job duration
Airflow task failures
Pipeline processing time
Data-quality failures
Record counts
```

19. Data Quality — 1 Minute

Say:

“Data quality is handled throughout the pipeline.”

Checks include:

```
Missing values
Duplicate IDs
Invalid data types
Invalid numeric ranges
Foreign-key integrity
Invalid statuses
Invalid timestamps
Row-count consistency
```

Examples:

```
order_id must be unique

order_value >= 0

delivery_distance_km >= 0

traffic_score between 0 and 100
```

Then:

"I also created automated tests under the tests directory for ingestion, transformations, data quality and pipeline contracts."

20. One Complete Data Flow Example — 2 Minutes

This is where you can demonstrate that you **actually understand how everything connects**.

Tell them:

"Let me explain one complete example using a GPS event."

Suppose a vehicle generates:

```
{
  "vehicle_id": "V101",
  "timestamp": "2026-08-31 10:30:00",
  "latitude": 13.08,
  "longitude": 80.27,
  "speed_kmph": 42
}
```

Then:

```
Vehicle
  ↓
GPS Producer
  ↓
Kafka gps_tracking topic
  ↓
Spark Structured Streaming
  ↓
Schema validation
  ↓
Transformation
  ↓
is_moving / signal analysis
  ↓
Processed/serving layer
  ↓
Analytics
  ↓
Power BI / Monitoring
```

Then use an order example.

“Similarly, suppose an order has a promised delivery time of 60 minutes and an actual delivery time of 75 minutes.”

Then:

```
delay = 75 - 60  
       = 15 minutes
```

Therefore:

```
is_delayed = 1
```

That information can then flow into:

```
fact_delivery  
  ↓  
dbt delivery_mart  
  ↓  
Power BI
```

And the business user sees:

“This region has an elevated delivery delay rate.”

That's the entire **raw-event-to-business-insight journey**.

21. Why Each Technology? — 1 Minute

If they ask:

“Why did you choose all these technologies?”

Answer:

Technology	Why
Python	Ingestion, automation, analytics, ML
Kafka	Real-time event streaming

Technology	Why
HDFS	Distributed data-lake storage
S3	Cloud object storage
Spark	Large-scale batch/stream processing
Hive	SQL/table layer over Hadoop data
Warehouse	Business-oriented analytical model
dbt	Modular SQL transformations/testing
Airflow	Pipeline orchestration
Scikit-learn	ML baseline models
Power BI	Business visualization
Prometheus	Metrics collection
Grafana	Monitoring dashboards
Docker	Local infrastructure
Git/GitHub	Version control

Then:

“The important thing is that I didn't choose the technologies independently. Each one has a role in the overall architecture.”

22. Biggest Technical Challenges — 2 Minutes

This section is extremely useful because interviewers often ask:

“What challenges did you face?”

Give 3–4.

Challenge 1 — Data Integration

“The first challenge was that the datasets have different grains and structures. For example, an order is one row per order while GPS is one row per vehicle observation. Joining them directly can create row multiplication. So I explicitly defined the grain of each dataset and used appropriate aggregation or time-based logic.”

Challenge 2 — Data Leakage

"The second challenge was machine-learning leakage. For delay prediction, actual delivery outcomes cannot be used as input features. So I explicitly excluded fields such as actual delivery time, delay minutes and final delay labels."

Challenge 3 — Real-Time Data

"The third challenge was that I don't have access to a real logistics company's production GPS stream. So I built the Kafka producer/consumer architecture and simulated bounded streaming events rather than claiming that I had a live production stream."

This is an **excellent honesty answer**.

Challenge 4 — Pipeline Dependencies

"Another challenge is dependency management. Analytics should only run after data processing is complete, and dbt models should only run when their source data is available. That's why I use Airflow to define dependencies."

23. Limitations — 1 Minute

If they ask:

"What would you improve?"

Say:

"There are several things I would improve before taking this to production."

Then:

1. Replace synthetic operational data with real production data.
2. Add stronger Kafka schema management using Avro or Protobuf.
3. Add Schema Registry.
4. Implement production-grade streaming checkpoints and state management.
5. Add Kafka consumer-lag monitoring.
6. Improve geospatial map matching.
7. Evaluate stronger ML models.
8. Add ML model monitoring.

9. Add CI/CD.
10. Add centralized secrets management.
11. Add data catalog and lineage.
12. Deploy the infrastructure properly on AWS or another cloud.

Then:

"So I consider the current project a complete portfolio-scale architecture and implementation prototype rather than a production deployment."

That's a **very mature answer**.

24. Future Scope — 1 Minute

Say:

"If I continued the project, I would add real-time ETA prediction, route optimization, inventory analytics, supplier analytics, carbon-emission tracking and automated anomaly detection."

Then:

"For example, instead of only predicting whether an order will be delayed, the next version could predict the expected arrival time and dynamically recommend alternative routes using real-time traffic."

25. Strong Closing — 30 Seconds

Finish with:

"So overall, LogiTrack demonstrates the complete lifecycle of logistics data.

I start with raw operational and external data, ingest it through batch APIs and Kafka, store it in the data-lake layer, process it using Spark, organize it using Hive and a dimensional warehouse, transform it through dbt, orchestrate the workflow with Airflow, perform SQL and Python analytics, build baseline ML models, expose business insights through Power BI, and monitor the infrastructure using Prometheus and Grafana.

The main thing I learned from the project is that a data project isn't just about writing queries or building dashboards. The important part is designing a reliable flow from **data generation to data ingestion, processing, modeling, analytics and finally business decision-making.**"